UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final Modelo

Disciplina: Qualidade de Software
Equipe: Victor Emanuel Silva de Queiroz - 567683 e Radlei Eugenio Doroth - 568379
Projeto: APM (Agent Package Manager): https://github.com/microsoft/apm
Fork: https://github.com/victtorqueiroz/apm
Pull Request: https://github.com/microsoft/apm/pull/3451
Commit SHA: 280b8a7b115b7a0c9b232467c376480c7cfde04e

1. Apresentação do projeto

O APM (Agent Package Manager) é um projeto open source da Microsoft que funciona como um gerenciador de pacotes para agentes de IA, tipo o Copilot e o Claude Code. Em vez de cada desenvolvedor configurar o agente na mão, o projeto lista num arquivo apm.yml o que o agente precisa (skills, prompts, plugins, etc.), e o comando apm install instala tudo. Assim, todo mundo da equipe fica com a mesma configuração.

O código é uma CLI em Python, na pasta src/apm_cli. Os testes usam pytest e ficam em tests/. A maior parte está em tests/unit, separada por partes do sistema, e há outras pastas para testes de integração, desempenho e aceitação. São 1.365 arquivos Python na pasta de testes, dos quais 1.174 analisados pelo TEMPY.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Conditional Test Logic	3288
Unknown Test	2434
Verbose Test	918
Programming Paradigms Blend	568
Exception Handling	555
Undefined Test	375

Test smell	Ocorrências detectadas
Redundant Print	160
Sleepy Test	100
Non-Functional Statement	83
Total	8481

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	34.818
Testes com falha (Failed)	69
Erros (Errors)	0
Ignorados (Skipped)	570
Cobertura total (%)	92,06

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	443.831
Complexidade ciclomática média das funções de teste	2,75 (36.668 funções)
Índice de manutenibilidade médio dos arquivos de teste	41,89 (1.365 arquivos)

3. Refatorações realizadas

3.1 Seleção das ocorrências

Escolhemos os quatro tipos de acordo com a severidade indicada no CATSPY: Conditional Test Logic, Unknown Test, Verbose Test e Exception Handling. Também pesou a quantidade de ocorrências no projeto, porque os quatro estão entre os mais frequentes no TEMPY. Resolvemos dividir assim:

- Victor: Exception Handling e Verbose Test (10 de cada)
- Radlei: Unknown Test e Conditional Test Logic (10 de cada)

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Exception Handling	tests/unit/test_mcp_overlays.py::test_validate_stdio_shell_string_error_includes_fix_it	b48a0fc	sim	sim	não
R02	Exception Handling	tests/unit/compilation/test_compilation.py::test_validation_error_display	9dd11ec	sim	sim	não
R03	Exception Handling	tests/unit/test_build_spec.py::test_spec_file_compiles_without_syntax_errors	b71eb00	sim	sim	não
R04	Exception Handling	tests/unit/test_runtime_detection.py::test_full_detection_workflow	e348537	sim	sim	não
R05	Exception Handling	tests/unit/test_script_runner.py::test_resolve_prompt_file_local_exists	f322392	sim	sim	não
R06	Exception Handling	tests/unit/test_script_runner.py::test_resolve_prompt_file_dependency_root	573c429	sim	sim	não
R07	Exception Handling	tests/unit/test_script_runner.py::test_resolve_prompt_file_dependency_subdirectory	1ad3a2f	sim	sim	não
R08	Exception Handling	tests/unit/test_script_runner.py::test_resolve_prompt_file_multiple_dependencies	8ae1373	sim	sim	não
R09	Exception Handling	tests/unit/test_script_runner.py::test_resolve_prompt_file_no_apm_modules	df2cc77	sim	sim	não
R10	Exception Handling	tests/unit/test_config_command.py::test_config_show_outside_project	dd1d4f0	sim	sim	não
R11	Verbose Test	tests/unit/integration/test_skill_integrator_hermetic.py::test_collision_warning_with_logger	69748b6	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R12	Verbose Test	tests/unit/integration/test_skill_integrator_phase3w4.py::test_collision_warning_with_logger	17ab795	sim	sim	não
R13	Verbose Test	tests/unit/integration/test_skill_integrator.py::test_native_skill_preserves_complete_structure	bdba062	sim	sim	não
R14	Verbose Test	tests/unit/integration/test_skill_integrator.py::test_force_overwrites_identical_bin	7c77500	sim	sim	não
R15	Verbose Test	tests/unit/deps/test_shared_clone_cache.py::test_short_sha_skips_tier1_and_resolves_via_tier2	aa05422	sim	sim	não
R16	Verbose Test	tests/unit/integration/test_hook_integrator.py::test_stale_sidecar_removed_on_sync	4fe7174	sim	sim	não
R17	Verbose Test	tests/unit/integration/test_hook_integrator_defect_regression.py::test_unambiguous_legacy_leaf_marker_migrates_to_canonical_owner	a8ddd23	sim	sim	não
R18	Verbose Test	tests/unit/deps/test_shared_clone_cache.py::test_bare_action_rmtrees_target_before_init	2516300	sim	sim	não
R19	Verbose Test	tests/unit/deps/test_shared_clone_cache.py::test_fetch_action_uses_explicit_url_not_origin	18ab66f	sim	sim	não
R20	Verbose Test	tests/unit/integration/test_hook_integrator.py::test_root_local_healer_preserves_dependency_source_entries	a461ebf	sim	sim	não
R21	Unknown Test	tests/unit/install/test_mcp_conflicts.py::test_all_defaults_no_error	14427e5	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R22	Unknown Test	tests/unit/install/test_mcp_conflicts.py::test_mcp_name_with_transport_stdio_no_url	1959ea6	sim	sim	não
R23	Unknown Test	tests/unit/install/test_mcp_conflicts.py::test_mcp_name_with_command_argv	990e1d1	sim	sim	não
R24	Unknown Test	tests/unit/install/test_mcp_conflicts.py::test_headers_with_url	bde14b2	sim	sim	não
R25	Unknown Test	tests/unit/test_view_command_cli_surface.py::test_no_project_root_skips_lockfile	b6d2ae8	sim	sim	não
R26	Unknown Test	tests/unit/test_view_command_cli_surface.py::test_rich_rendering_no_context_files	aa9dd27	sim	sim	não
R27	Unknown Test	tests/unit/test_view_command_cli_surface.py::test_import_error_fallback_text	d76b7c5	sim	sim	não
R28	Unknown Test	tests/unit/install/test_cache_pin.py::test_write_marker_silently_skips_missing_dir	7eda87a	sim	sim	não
R29	Unknown Test	tests/unit/integration/test_mcp_integrator.py::test_noop_for_unknown_server	9251edd	sim	sim	não
R30	Unknown Test	tests/unit/core/test_auth_phase3.py::test_empty_host_is_noop	c5b1673	sim	sim	não
R31	Conditional Test Logic	tests/unit/test_auth.py::test_ado_git_env_strips_raw_github_token_sources	d7216a0	sim	sim	não
R32	Conditional Test Logic	tests/unit/test_auth.py::test_gitlab_git_env_uses_safe_basic_header	a092840	sim	sim	não
R33	Conditional Test Logic	tests/unit/test_auth.py::test_concurrent_same_host_emits_once	6e2b95b	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R34	Conditional Test Logic	tests/unit/compilation/test_coverage_guarantees.py::test_hierarchical_coverage_verification	5072a1b	sim	sim	sim: Programming Paradigms Blend
R35	Conditional Test Logic	tests/unit/compilation/test_coverage_guarantees.py::test_coverage_constraint_priority_over_efficiency	af13210	sim	sim	não
R36	Conditional Test Logic	tests/unit/integration/test_data_driven_dispatch.py::test_skills_is_multi_target	9d615ff	sim	sim	não
R37	Conditional Test Logic	tests/unit/integration/test_data_driven_dispatch.py::test_dispatch_entries_have_valid_methods	50b8dc1	sim	sim	não
R38	Conditional Test Logic	tests/unit/integration/test_hook_integrator.py::test_filter_generic_hooks_universal	ff5460b	sim	sim	não
R39	Conditional Test Logic	tests/unit/core/test_public_github_anonymous_first.py::test_private_github_file_fetch_uses_one_scoped_fallback	69c49fe	sim	sim	não
R40	Conditional Test Logic	tests/unit/integration/test_data_driven_dispatch.py::test_every_target_primitive_has_dispatch_path	5b6f8f0	sim	sim	não

3.3 Exemplos detalhados

[R01] - [Exception Handling]

([tests/unit/test_mcp_overlays.py::test_validate_stdio_shell_string_error_includes_fix_it])

Antes:

```python
def test_validate_stdio_shell_string_error_includes_fix_it(self):
    """Error message must include the corrected command/args shape."""
    try:
        MCPDependency.from_dict(
            {
                "name": "fix-it",
                "registry": False,
                "transport": "stdio",
                "command": "npx -y some-pkg",
            }
        )
    except ValueError as e:
        msg = str(e)
        assert "command: npx" in msg
        assert '"-y"' in msg
        assert '"some-pkg"' in msg
    else:
        pytest.fail("expected ValueError")
```

Depois:

```python
def test_validate_stdio_shell_string_error_includes_fix_it(self):
    """Error message must include the corrected command/args shape."""
    with pytest.raises(ValueError) as exc_info:
        MCPDependency.from_dict(
            {
                "name": "fix-it",
                "registry": False,
                "transport": "stdio",
                "command": "npx -y some-pkg",
            }
        )
    msg = str(exc_info.value)
    assert "command: npx" in msg
    assert '"-y"' in msg
    assert '"some-pkg"' in msg
```

Por que era um test smell: O teste queria verificar que a função lança “ValueError”, porém fazia isso manualmente, com “try/except/else” e um “pytest.fail” no “else”. Ou seja, facilmente poderia ocorrer um erro: caso alguém esquecesse o “else”, o teste passaria mesmo sem exceção alguma. Além disso, como os assert ficavam dentro do except, quando um deles

falhava, o pytest mostrava duas exceções encadeadas, o que deixava o erro mais confuso de ler.

Estratégia do CATSPY aplicada: O CATSPY recomenda usar o recurso do próprio framework para verificar exceções (assertRaises, no unittest). Como o projeto usa pytest, apliquei o equivalente: with pytest.raises(ValueError) as exc_info. Precisei adaptar uma coisa que o exemplo do catálogo não mostra: este teste também confere a mensagem do erro. Então os assert foram para depois do bloco with, lendo str(exc_info.value), porque dentro do bloco eles nunca seriam executados. O else com pytest.fail foi removido, porque o pytest.raises já reprova o teste se a exceção não for lançada.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	6	4
SLOC do arquivo de teste (raw_por_arquivo_e_total)	735	732

[R11] - [Verbose Test]

([tests/unit/integration/test_skill_integrator_hermetic.py::test_collision_warning_with_logger])

Antes:

```python
def test_collision_warning_with_logger(self, tmp_path: Path) -> None:
    pkg_dir = tmp_path / "my-skill"
    pkg_dir.mkdir()
    skill_md = pkg_dir / "SKILL.md"
    skill_md.write_text("# skill", encoding="utf-8")

    target = _make_target(name="copilot", root_dir=".github", auto_create=True)
    # Pre-create the target skill dir
    target_skill_dir = tmp_path / ".github" / "skills" / "my-skill"
    target_skill_dir.mkdir(parents=True)
    (target_skill_dir / "SKILL.md").write_text("# old", encoding="utf-8")

    logger = MagicMock()
    dep_ref = MagicMock()
    dep_ref.get_unique_key = MagicMock(return_value="other/repo")
    pi = _make_package_info(pkg_dir)
    pi.dependency_ref = dep_ref

    integrator = SkillIntegrator()

    with patch.object(
        SkillIntegrator,
        "_build_ownership_maps",
        return_value=(
            {},
            {target_skill_dir.relative_to(tmp_path).as_posix(): "prev-owner/pkg"},
        ),
    ):
        result = integrator._integrate_native_skill(
            pi,
            tmp_path,
            skill_md,
            logger=logger,
            targets=[target],
        )
    logger.warning.assert_called_once_with(
        "Skill 'my-skill' from 'other/repo' replaced "
        "'prev-owner/pkg' -- remove one package to avoid this"
```

```python
        logger.warning.assert_called_once_with(
            "Skill 'my-skill' from 'other/repo' replaced "
            "'prev-owner/pkg' -- remove one package to avoid this"
        )
        assert result.skill_updated is True
        assert result.skill_created is False
        assert result.skill_skipped is False
        assert result.skill_path == target_skill_dir / "SKILL.md"
        assert result.target_paths == [target_skill_dir]
        assert result.references_copied == 1
        assert (target_skill_dir / "SKILL.md").read_bytes() == skill_md.read_bytes() == b"# skill"
```

Depois:

(Método)

```python
def test_collision_warning_with_logger(self, tmp_path: Path) -> None:
    pi, skill_md, target_skill_dir = _make_colliding_skill(tmp_path)
    target = _make_target(name="copilot", root_dir=".github", auto_create=True)
    logger = MagicMock()
    previous_owners = {target_skill_dir.relative_to(tmp_path).as_posix(): "prev-owner/pkg"}

    with patch.object(
        SkillIntegrator, "_build_ownership_maps", return_value=({}, previous_owners)
    ):
        result = SkillIntegrator()._integrate_native_skill(
            pi, tmp_path, skill_md, logger=logger, targets=[target]
        )
    logger.warning.assert_called_once_with(
        "Skill 'my-skill' from 'other/repo' replaced "
        "'prev-owner/pkg' -- remove one package to avoid this"
    )
    assert result.skill_updated is True
    assert result.skill_created is False
    assert result.skill_skipped is False
    assert result.skill_path == target_skill_dir / "SKILL.md"
    assert result.target_paths == [target_skill_dir]
    assert result.references_copied == 1
    assert (target_skill_dir / "SKILL.md").read_bytes() == skill_md.read_bytes() == b"# skill"
```

(Função Auxiliar)

```python
def _make_colliding_skill(tmp_path: Path):
    """Create a package skill plus an existing target skill owned by another package."""
    pkg_dir = tmp_path / "my-skill"
    pkg_dir.mkdir()
    skill_md = pkg_dir / "SKILL.md"
    skill_md.write_text("# skill", encoding="utf-8")

    target_skill_dir = tmp_path / ".github" / "skills" / "my-skill"
    target_skill_dir.mkdir(parents=True)
    (target_skill_dir / "SKILL.md").write_text("# old", encoding="utf-8")

    dep_ref = MagicMock()
    dep_ref.get_unique_key.return_value = "other/repo"
    pi = _make_package_info(pkg_dir, dep_ref=dep_ref)
    return pi, skill_md, target_skill_dir
```

Por que era um test smell: O método tinha 45 linhas, e mais da metade só preparava o cenário: criava o pacote com o SKILL.md, uma skill antiga no destino para provocar a colisão e um dono diferente. A parte que importa, a chamada e os assert, ficava escondida no fim. Quem lia o teste precisava passar por toda a preparação para entender o que ele verificava.

Estratégia do CATSPY aplicada: O CATSPY sugere dividir o teste em testes menores, um para cada responsabilidade. Aqui isso não fazia sentido, porque o teste só verifica uma coisa (o aviso de colisão). O excesso de linhas vinha da preparação. Então adaptei a estratégia e usei Extract Method: a preparação foi para a função auxiliar _make_colliding_skill, que devolve o que o teste usa depois. O teste caiu de 45 para cerca de 23 linhas e ficou com as três partes visíveis: preparar, executar e verificar. Os assert não mudaram.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	8	8
SLOC do arquivo de teste (raw_por_arquivo_e_total)	833	826

[R25] – [Unknown Test] ([tests/unit/test_view_command_cli_surface.py::test_no_project_root_skips_lockfile])

Antes:

```
> cd /home/radlei/Documentos/apm
git show bód2ae8f~1:tests/unit/test_view_command_cli_surface.py | grep -n -A 20 "def test_no_project_root_skips_lockfile"
242:    def test_no_project_root_skips_lockfile(self, tmp_path):
243-        from apm_cli.commands.view import display_package_info
244-
245-        logger = _make_logger()
246-        pkg_info = _make_package_info()
247-
248-        with patch(
249-            "apm_cli.commands.view._get_detailed_package_info",
250-            return_value=pkg_info,
251-        ):
252-            # No project_root passed -> _lookup_lockfile_ref not called
253-            display_package_info("org/repo", tmp_path, logger, project_root=None)
254-
255-
256-# ---------------------------------------------------------------------------
257-# _display_marketplace_plugin
258-# ---------------------------------------------------------------------------
259-
260-
261-class TestDisplayMarketplacePlugin:
262-    def _build_plugin(self, *, source=None, version="1.0.0", tags=None, description="Desc"):
```

Depois:

```
> git show bód2ae8f:tests/unit/test_view_command_cli_surface.py | grep -n -A 24 "def test_no_project_root_skips_lockfile"
242:    def test_no_project_root_skips_lockfile(self, tmp_path):
243-        from apm_cli.commands.view import display_package_info
244-
245-        logger = _make_logger()
246-        pkg_info = _make_package_info()
247-
248-        with (
249-            patch(
250-                "apm_cli.commands.view._get_detailed_package_info",
251-                return_value=pkg_info,
252-            ),
253-            patch("apm_cli.commands.view._lookup_lockfile_ref") as lookup,
254-        ):
255-            # No project_root passed -> _lookup_lockfile_ref not called
256-            display_package_info("org/repo", tmp_path, logger, project_root=None)
257-
258-        lookup.assert_not_called()
259-
260-
261-# ---------------------------------------------------------------------------
262-# _display_marketplace_plugin
263-# ---------------------------------------------------------------------------
264-
265-
266-class TestDisplayMarketplacePlugin:
```

Por que era um test smell: O teste não tinha nenhuma asserção, então passava mesmo que o comportamento descrito no comentário (não chamar `_lookup_lockfile_ref`) deixasse de existir.

Estratégia do CATSPY aplicada: [Introduzir uma asserção que valide o resultado esperado. Foi preciso adaptar: os exemplos do catálogo conferem um estado alterado, e aqui o efeito esperado é uma chamada que não deve acontecer. Usei `patch` e `lookup.assert_not_called()`, e confirmei que o teste falha quando `project_root` é informado.]

Métricas do teste refatorado:

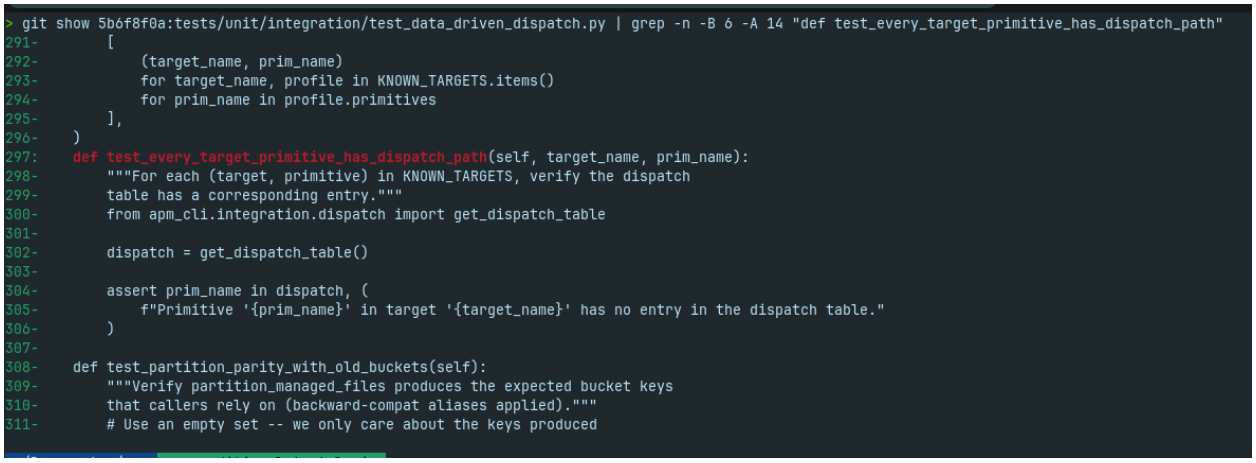
Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	418	433

[R40] – [Conditional Test Logic] ([tests/unit/integration/test_data_driven_dispatch.py::test_every_target_primitive_has_dispatch_path])

Antes:

```
> git show 5b6f8f0a~1:tests/unit/integration/test_data_driven_dispatch.py | grep -n -B 3 -A 20 "def test_every_target_primitive_has_dispatch_path"
286-class TestExhaustivenessChecks:
287-    """Structural checks ensuring no target x primitive pair is orphaned."""
288-
289:    def test_every_target_primitive_has_dispatch_path(self):
290-        """For each (target, primitive) in KNOWN_TARGETS, verify the dispatch
291-        table has a corresponding entry."""
292-        from apm_cli.integration.dispatch import get_dispatch_table
293-
294-        dispatch = get_dispatch_table()
295-
296-        for target_name, profile in KNOWN_TARGETS.items():
297-            for prim_name in profile.primitives:
298-                assert prim_name in dispatch, (
299-                    f"Primitive '{prim_name}' in target '{target_name}' has no "
300-                    f"entry in the dispatch table."
301-                )
302-
303-    def test_partition_parity_with_old_buckets(self):
304-        """Verify partition_managed_files produces the expected bucket keys
305-        that callers rely on (backward-compat aliases applied)."""
306-        # Use an empty set -- we only care about the keys produced
307-        buckets = BaseIntegrator.partition_managed_files(set())
308-
309-        # Expected keys from the old hardcoded version:
```

Depois:



```
> git show 5b6f8f0a:tests/unit/integration/test_data_driven_dispatch.py | grep -n -B 6 -A 14 "def test_every_target_primitive_has_dispatch_path"
291-        [
292-            (target_name, prim_name)
293-            for target_name, profile in KNOWN_TARGETS.items()
294-            for prim_name in profile.primitives
295-        ],
296-    )
297:    def test_every_target_primitive_has_dispatch_path(self, target_name, prim_name):
298-        """For each (target, primitive) in KNOWN_TARGETS, verify the dispatch
299-        table has a corresponding entry."""
300-        from apm_cli.integration.dispatch import get_dispatch_table
301-
302-        dispatch = get_dispatch_table()
303-
304-        assert prim_name in dispatch, (
305-            f"Primitive '{prim_name}' in target '{target_name}' has no entry in the dispatch table."
306-        )
307-
308-    def test_partition_parity_with_old_buckets(self):
309-        """Verify partition_managed_files produces the expected bucket keys
310-        that callers rely on (backward-compat aliases applied)."""
311-        # Use an empty set -- we only care about the keys produced
```

Por que era um test smell: Dois laços aninhados verificavam 46 combinações numa única execução, e uma falha não dizia qual combinação estava errada.

Estratégia do CATSPY aplicada: Separar os casos em testes distintos. O catálogo mostra só `if/else`, então adaptei usando `@pytest.mark.parametrize` sobre os pares (target, primitiva), que gera um teste por combinação com a mesma asserção. Conferi que os 46 casos continuam sendo testados.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	4	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	690	699

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	8.481	8.421	−60
Testes aprovados	34.818	34.884	+66
Testes com falha	69	69	0
Cobertura total (%)	92,06	92,06	0,00 p.p.
SLOC dos testes	443.831	443.768	−63
Complexidade ciclomática média das funções de teste	2,75	2,75	0,00

Métrica	Antes	Depois	Variação
Índice de manutenibilidade médio dos arquivos de teste	41,89	41,89	0,00

Test smell	Antes	Depois	Variação
[Conditional Test Logic]	3.288	3.259	−29
[Unknown Test]	2.434	2.424	−10
[Verbose Test]	918	906	-12
[Exception Handling]	555	545	-10
Total	7.195	7.134	-61

5. Análise dos resultados

Com base nas tabelas das Seções 2, 3 e 4, responda:

Quais métricas melhoraram e quais pioraram?

Melhoraram o total de test smells (de 8.481 para 8.421, −60), com queda nos quatro tipos refatorados (Conditional Test Logic −29, Unknown Test −10, Verbose Test −12 e Exception Handling −10), e o SLOC dos testes (de 443.831 para 443.768, −63).

A complexidade ciclomática média (2,75) e o índice de manutenibilidade médio (41,89) ficaram iguais, porque a mudança é pequena diante de mais de 36 mil funções de teste.

Pioraram o número de testes, que subiu em 66 por causa do `parametrize` (esperado, e o monitor aceitou), e o Programming Paradigms Blend, que subiu de 568 para 569.

Algumas quedas vieram como efeito colateral de outras refatorações: o Verbose Test caiu 12, e não 10, porque duas refatorações de Conditional Test Logic em test_coverage_guarantees.py também encurtaram os testes. E a refatoração de Verbose Test em test_short_sha_skips_tier1_and_resolves_via_tier2 também tirou um Conditional Test Logic, porque o if do git falso foi para a função auxiliar.

Alguma refatoração introduziu novos test smells? Quais?

Sim, o Programming Paradigms Blend em `tests/unit/compilation/test_coverage_guarantees.py` (de 0 para 1). Ele veio da refatoração R34, que criou uma função auxiliar solta no módulo, num arquivo cujos testes ficam em uma classe, misturando código procedural e orientado a objetos.

As refatorações de Exception Handling e Verbose Test não introduziram smells novos. Dois testes de Exception Handling (R02 e R03) continuam marcados como Unknown Test, mas já tinham esse smell antes.

O resultado dos testes ou a cobertura mudou?

Os testes aprovados passaram de 34.818 para 34.884 (+66), por causa dos testes novos gerados pelo `parametrize` e pela divisão de testes. As mesmas 69 falhas da linha de base continuam (comparamos as duas listas e são idênticas), e os 570 ignorados não mudaram. Nenhum teste que passava antes passou a falhar. A cobertura ficou em 92,06% nas duas medições, o que era esperado, já que só código de teste foi alterado

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

O Unknown Test foi o mais mecânico: bastava acrescentar uma asserção. A dificuldade foi escolher uma que fizesse sentido, porque várias funções só lançam erro ou devolvem `None`, e nesses casos a única verificação possível é fraca. O Conditional Test Logic foi o mais trabalhoso, porque o TEMPY analisa linhas e não a estrutura do código: em alguns testes ele continuou acusando o smell depois da refatoração (por exemplo, quando o `for` só mudava de lugar para um helper), e esses casos não foram commitados.

Entre os outros dois, o Exception Handling foi mais fácil: depois de entender o pytest.raises e as fixtures tmp_path e monkeypatch, os casos se repetiam. O Verbose Test deu mais trabalho, porque cada teste era de um jeito e foi preciso decidir o que extrair para a função auxiliar e o que deixar visível no teste. Também percebemos que, no Verbose Test, a complexidade ciclomática quase não muda, porque a preparação extraída não tem if nem for. O ganho aparece no tamanho dos métodos, e não nas médias.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Para cada smell, lemos a definição e a refatoração indicada na página do catálogo antes de alterar o teste, e registramos a página consultada no diário.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

O exemplo refatorado, pois ele mostrou a forma esperada da correção e ajudou bastante.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

A página do Conditional Test Logic mostra só `if/else`, mas no APM o smell aparecia quase sempre como `for`. Usamos `@pytest.mark.parametrize`, que cria um teste por caso. Também houve casos de concorrência (`pool.map`) que a página não cobre. Em Unknown Test, os exemplos mudam um estado e conferem o resultado, mas várias funções do APM só lançam erro ou devolvem `None`.

No Exception Handling, o catálogo mostra try/except com assertRaises (unittest), e usamos o equivalente do pytest, pytest.raises. Mas 6 casos eram try/finally usados só para voltar à pasta original. Neles, trocamos o try pelas fixtures tmp_path e monkeypatch.chdir. No Verbose Test, o catálogo sugere dividir o teste, mas quase todos testavam uma coisa só e eram longos por causa da preparação. Então extraímos a preparação para funções auxiliares.

O que faltou ou poderia ser melhorado no catálogo?

Exemplos com laços `for`, `parametrize` e mocks, a opção de extrair a preparação para uma função auxiliar no Verbose Test e uma orientação sobre falsos positivos do detector.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Victor	R01 a R20	20
Radlei	R21 a R40	20

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

- Um teste de shell interativo travava o terminal (`suspended (tty input)`). Rodamos em segundo plano, sem teclado.
- O `.venv/bin` precisa estar no PATH, senão 362 testes que usam o binário `apm` são ignorados.
- Os scripts da professora foram atualizados depois da primeira medição, e refizemos o "antes" num clone limpo.
- Falsos positivos do TEMPY (`pytest.raises` marcado como Unknown Test, `sleep` mockado) e o fato de ele olhar linhas, não a estrutura do código.
- No Windows, 3 testes falham por causa de caminhos no estilo Linux, e test_script_runner.py dá erro de importação circular quando roda sozinho. Os dois problemas já existiam.
- O projeto é grande e era desconhecido para nós (visto que é open source). Foi preciso focar só no teste, sem entender tudo o que a função testada faz.
- Pouca prática com pytest (pytest.raises, fixtures como tmp_path e monkeypatch)
- Na PR, o lint do projeto (Ruff) apontou problemas de estilo que o pytest não pegava, e o main do projeto tinha avançado, o que gerou um conflito ao trazer os commits.

O que você aprendeu com a atividade?

Aprendemos que um teste que passa nem sempre verifica alguma coisa. Vários testes do APM só chamavam uma função e confiavam em "não deu erro", e foi preciso entender o código para escolher uma verificação que fizesse algum sentido.

Também aprendemos a identificar os testes em preparação, execução e verificação, e a reconhecer também o que deixa um teste longo ou frágil. Além disso, que refatorar não pode mudar o que o teste verifica, e que pra provar isso vale fazendo o teste falhar de propósito. Entendemos também a nem sempre confiar cegamente na ferramenta de detecção, e sim conferir caso a caso. E relembramos também como é trabalhar com um projeto open source que exige todo um conjunto de etapas.

9. Contribuição (Pull Request)

Abrimos a issue microsoft/apm#3450 e, em seguida, a PR microsoft/apm#3451 no repositório original, como pede o CONTRIBUTING.md do projeto. A PR tem as 40 refatorações (10 de cada tipo), em 21 arquivos de tests/, sem código de produção e sem as pastas de métricas.

Como o main do projeto estava 451 commits à frente da base (280b8a7), criamos a branch da PR a partir do main atual e trouxemos os commits com git cherry-pick. Houve um conflito de import em test_data_driven_dispatch.py, resolvido mantendo os dois imports. Os testes dos 21 arquivos passaram (1579), exceto 3 que já falhavam antes no Windows.

Depois de aberta, a revisão automática do Copilot e o lint do projeto (Ruff) apontaram alguns problemas apenas de estilização, os quais foram corrigidos com um commit extra. Também assinamos o CLA da Microsoft. Até a entrega, nenhum mantenedor respondeu, e o CI aguarda autorização para rodar.

10. Observações gerais

- O TEMPY e o CATSPY usam critérios diferentes: o TEMPY marca qualquer try como Exception Handling, conta linhas em branco no Verbose Test (limite de 45, contra 30 no CATSPY) e não reconhece pytest.raises como verificação. Encontramos também um falso positivo (_make_ado_downloader).
- No Verbose Test, a complexidade ciclomática quase não muda. O ganho aparece no tamanho dos métodos.
- As medições foram feitas na refatoracao-test-smells, e a PR saiu de outra branch, para não levar as métricas junto.
- Usamos o Claude Code como assistente.
- Sugestão para o CATSPY: o catálogo poderia ter uma parte sobre falsos positivos, ou seja, casos que as ferramentas marcam mas que não são o smell de verdade. Encontramos vários: um try/finally usado só para voltar à pasta original (marcado como Exception Handling), uma função auxiliar marcada como Verbose Test, testes com pytest.raises marcados como Unknown Test e sleep já mockado marcado como Sleepy Test. Saber isso de antemão ajudaria a escolher as ocorrências certas.